

Loan Approval Prediction System

A Simple, Easy-to-Understand Project Overview

Built with Python, SQL, Pandas, Scikit-learn, and Power BI

This document explains the project in plain, simple language -
no technical background needed to follow along.

1. What Is This Project?

This project looks at a loan applicant's details - like income, credit score, job status, and how much loan they want - and helps decide:

- Should the loan be **Approved** or **Rejected**?
- If approved, **how much money** should actually be given?
- If rejected, **what is the reason**, in plain words?

In short: it takes messy, real-world applicant data and turns it into a clear, explainable loan decision - the same way a bank would think about it, but automated.

2. How It Works (Step by Step)

Think of it like an assembly line. Each step improves the data a little more:

- **Step 1 - Get the data:** A list of loan applicants and their details.
- **Step 2 - Clean the data:** Fix missing values, typos, and duplicate entries.
- **Step 3 - Store it in SQL:** Organize the data so it can be searched easily.
- **Step 4 - Create useful new info:** Like monthly EMI, or income left after EMI.
- **Step 5 - Train the model:** Show the computer thousands of past examples so it learns the pattern.
- **Step 6 - Test the model:** Check how good its guesses are.
- **Step 7 - Pick the best model:** Out of 3 models tried, keep the one that guessed best.
- **Step 8 - Save the model:** So it can be reused without retraining every time.
- **Step 9 - Predict new applicants:** Feed in a new person's details, get a decision.
- **Step 10 - Send data to Power BI:** So managers can see charts, not just numbers.

3. Tools Used (In Simple Words)

- **Python** - the programming language used to clean data and build the model.
- **Pandas** - a tool for organizing data, like a smart spreadsheet inside Python.
- **SQL (SQLite)** - stores the applicant data so it can be searched and filtered, like a filing cabinet.
- **Scikit-learn** - the library used to train and compare machine learning models.
- **Power BI** - turns the results into charts and dashboards for business people to view.

4. What Is "Machine Learning" Doing Here?

Imagine showing a child 1,000 old loan applications, and for each one telling them "this got approved" or "this got rejected." After seeing enough examples, the child starts noticing patterns - like "people with bad credit history usually get rejected."

That is exactly what the model does. It studies thousands of past examples, learns the pattern, and then uses that pattern to guess the outcome for a brand-new applicant it has never seen before.

5. About the Data

- 5,000 sample loan applicant records were used.
- Details included: Gender, Marital Status, Education, Employment Status, Income.
- Also included: Loan Amount, Loan Term, Credit Score, Credit History, Existing Loans.
- The answer column: Loan Status - Approved or Rejected.
- The data was made realistic on purpose - with missing values, duplicate entries, and messy text - just like real bank data.

6. Cleaning the Data & Creating Useful Info

Cleaning the data:

- Filled in missing values (most common value for categories, middle value for numbers).
- Fixed inconsistent text, like extra spaces or mixed capital/small letters.
- Removed duplicate applications.
- Fixed values that were clearly out of a normal range.

New useful columns created:

- **Total Income** - applicant's income plus co-applicant's income.
- **EMI** - the estimated monthly loan payment.
- **Balance Income** - how much income is left after paying the EMI.
- **Debt-to-Income Ratio** - how much of the income already goes toward debt.
- **Risk Category** - a simple label: Low, Medium, or High risk.

7. The 3 Models Compared

- **Logistic Regression** - the simplest method. It looks at each factor (income, credit score, etc.), gives it a weight, and adds everything up to decide. Easy to explain.
- **Decision Tree** - works like a flowchart of yes/no questions. "Is credit history good? -> Yes -> Is income above a limit? -> Yes -> Approve."
- **Random Forest** - instead of one flowchart, it builds many flowcharts and lets them vote. Usually stronger, but harder to explain simply.

All three were trained and tested on the exact same data, so the comparison is fair.

8. Results - Which Model Won?

Each model was scored using 4 simple checks: Accuracy, Precision, Recall, and F1-Score (explained below the table).

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.73	0.73	0.87	0.80
Decision Tree	0.69	0.72	0.83	0.77
Random Forest	0.71	0.71	0.88	0.79

(Green row = the winning model: Logistic Regression)

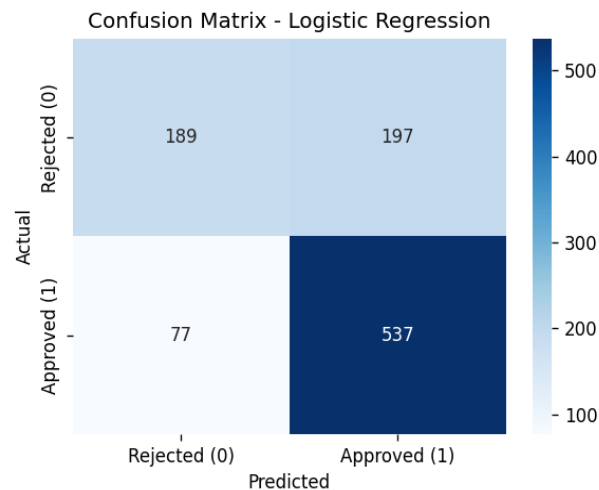
What do these words mean?

- **Accuracy** - out of 100 guesses, how many were correct overall?
- **Precision** - out of everyone the model said "approve," how many really deserved it? (Are we being too generous?)
- **Recall** - out of everyone who really deserved approval, how many did the model catch? (Are we rejecting good people by mistake?)
- **F1-Score** - one single number that balances Precision and Recall together.

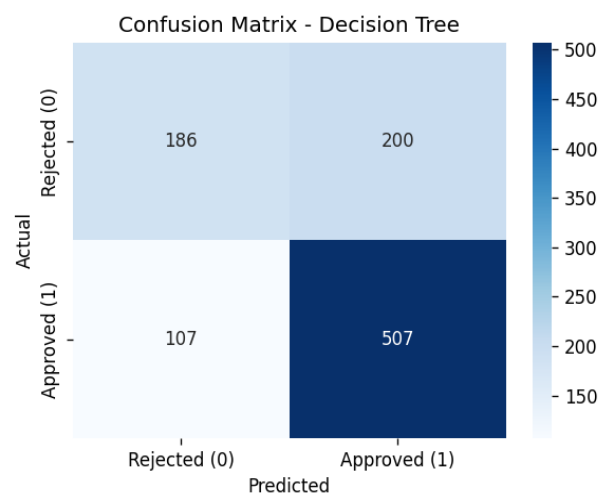
Logistic Regression was chosen as the best model because it had the highest F1-Score (80%) - meaning it best balanced approving good applicants and catching risky ones.

9. How Right or Wrong Was Each Model?

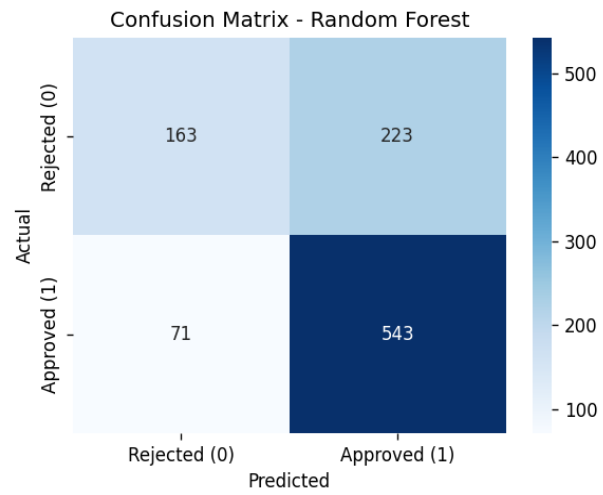
A "confusion matrix" is a simple picture that shows how many predictions were correct and how many were wrong, for each model.



Logistic Regression



Decision Tree



Random Forest

10. How the Final Decision Is Made

The model alone only says "approve" or "reject." That's not enough for a real bank, so a second, simple step was added:

- **Step 1:** The model predicts if the applicant is trustworthy (Approve / Reject).
- **Step 2:** A simple rule checks how much loan the applicant can actually afford, based on their income.
- **Result:** Approved (full amount), Partially Approved (smaller amount), or Rejected.
- If rejected, the system lists the real reasons - low credit score, high debt, unemployment, too many existing loans, etc. - instead of just saying "no."
- All loan amounts are shown in Rupees, for example: Rs 1,20,000.

11. The Power BI Dashboard

All the results are also saved into one clean file that Power BI can open directly. The dashboard shows:

- Approval rate and Rejection rate
- Risk category breakdown (Low / Medium / High)
- Credit score analysis
- Income analysis
- Loan amount trends (requested vs. approved)
- Applicant profile summary

This lets managers and business teams explore the results visually, without touching any code.

12. What Did We Learn? (Business Insights)

- Credit history is the single strongest sign of whether a loan gets approved.
- A high debt-to-income ratio often leads to rejection, even for small loan amounts.
- Unemployed applicants are rejected more often, even with similar income to others.
- The affordability check catches cases where a good applicant asks for more money than they can actually repay.

13. Conclusion

- A complete, working loan approval system was built - from raw data to final decision.
- Three machine learning models were compared, and the best one was chosen fairly.
- The winning model, **Logistic Regression**, correctly predicts loan outcomes 73% of the time on new data it has never seen.
- A real-world decision layer was added: approved amount + rejection reason, not just yes/no.
- A live Power BI dashboard was delivered for business reporting.
- The whole system is saved and ready to reuse - no need to retrain from scratch each time.

This project shows how a real lending decision can be automated end-to-end - combining data cleaning, machine learning, business rules, and reporting into one simple system.